
Introducción al Aprendizaje Automático

1er cuatrimestre 2025



Universidad
Nacional
de San Martín

Métricas para clasificación

El objetivo de esta presentación es introducir las métricas más comunes para evaluar el rendimiento de un modelo de clasificación.

¿Qué podemos usar para evaluar una clasificación?

Matriz de Confusión (Ojo, ¡no es una métrica!)

Exactitud

Precisión/Exhaustividad

F-Score

Clasificación Binaria

Problema general: separar los elementos de un conjunto en **dos grupos** bajo cierta/s regla/s de clasificación.

Clasificación Binaria

Problema general: separar los elementos de un conjunto en **dos grupos** bajo cierta/s regla/s de clasificación.

Ejemplos:



Examen médico

Enfermo / No enfermo
Test de embarazo



Educación

Aprobado / No
aprobado



Bromatología

Apto / No apto para
consumo



Control calidad

Seguro / No seguro

Clasificación Binaria

En general, nos interesa un grupo en particular.

Clasificación Binaria

¿Qué puede pasar con **un** test?
(ejemplo: test de embarazo)

Clasificación Binaria

¿Qué puede pasar con **un** test?
(ejemplo: test de embarazo)

Verdadero positivo (acierto)

test positivo, paciente embarazada

Verdadero negativo (acierto)

test negativo, paciente **no** embarazada

Clasificación Binaria

¿Qué puede pasar con **un** test?
(ejemplo: test de embarazo)

Verdadero positivo (acierto)

test positivo, paciente embarazada

Verdadero negativo (acierto)

test negativo, paciente no embarazada

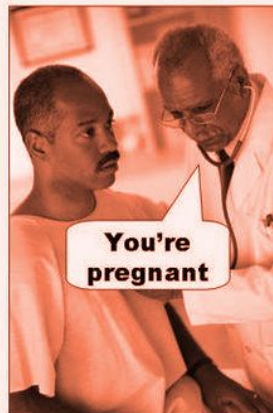
Falso positivo (error tipo 1)

test positivo, paciente **no** embarazada

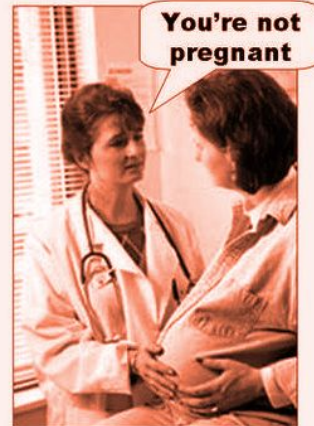
Falso negativo (error tipo 2)

test negativo, paciente embarazada

Type I error
(false positive)



Type II error
(false negative)



Clasificación Binaria • Matriz de confusión

Cuando aplicamos ese test a una población:

		Clase Predicha	
		Clase 1	Clase 2
Clase Verdadera	Clase 1	Elementos de la clase 1 correctamente identificados	Elementos de la clase 1 identificados como clase 2
	Clase 2	Elementos de la clase 2 identificados como clase 1	Elementos de la clase 2 correctamente identificados

¡Tiene toda la información que necesitamos!

Clasificación Binaria • Matriz de confusión

Cuando aplicamos ese test a una población:

		Clase Predicha	
		Positivos	Negativos
Clase Verdadera	Positivos	Verdaderos Positivos (VP o TP)	Falsos Negativos (FN)
	Negativos	Falsos Positivo (FP)	Verdaderos Negativos (VN o TN)

¡Tiene toda la información que necesitamos!

Clasificación Binaria • Matriz de confusión

Cuando aplicamos ese test a una población:

		Clase Predicha	
		Positivos	Negativos
Clase Verdadera	Positivos	Verdaderos Positivos (VP o TP)	Falsos Negativos (FN)
	Negativos	Falsos Positivo (FP)	Verdaderos Negativos (VN o TN)

Por ejemplo, la **exactitud** se calcula como la fracción de los datos correctamente clasificados :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Clasificación Binaria • Matriz de confusión

Ejemplo: Titanic

		Clase Predicha	
		No Sobrevivieron	Sobrevivieron
Clase Verdadera	No Sobrevivieron	372	177
	Sobrevivieron	119	223

Modelo: Si es pasajero de 3ra clase entonces 'no sobrevivió'.

Clasificación Binaria • Matriz de confusión

Ejemplo: Titanic

		Clase Predicha		Resultado
		No Sobrevivieron	Sobrevivieron	
Clase Verdadera	No Sobrevivieron	372	177	549
	Sobrevivieron	119	223	342
	Total pasajeros			891

Ejercicio: para la clase “sobrevivieron”, indicar Verdaderos Positivos, Falsos Positivos, Falsos Negativos y Verdaderos Negativos. Calcular la exactitud.

Clasificación Binaria • Matriz de confusión

Ejemplo: Titanic

		Clase Predicha	
		No Sobrevivieron	Sobrevivieron
Clase Verdadera	No Sobrevivieron	372	177
	Sobrevivieron	119	223

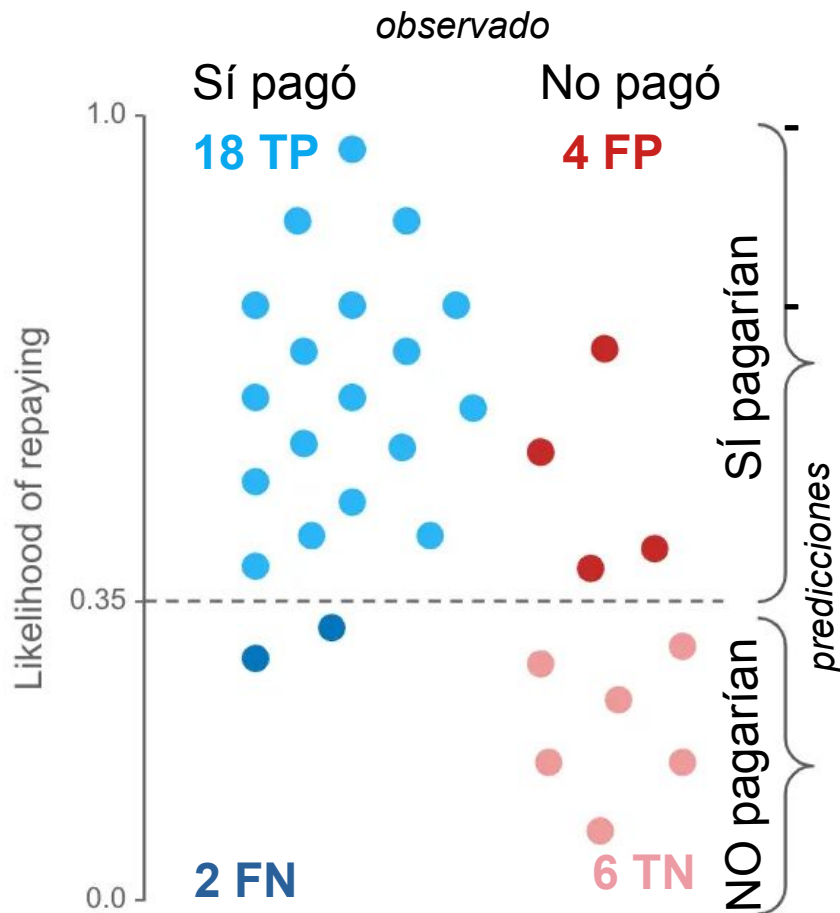
Exactitud = Aciertos/Total = $(VP + VN) / (VP + VN + FP + FN) = (372 + 223) / (372 + 223 + 119 + 177)$
= 0.668

Clasificación Binaria • Exactitud

La exactitud puede ser una métrica engañosa:

- En problemas desbalanceados, podemos obtener exactitudes muy altas con clasificadores triviales.
- No da información sobre los tipos de errores que estamos cometiendo.
 - En muchos problemas, una de las clases nos interesa más que la otra.

Clasificación Binaria • Precisión y Exhaustividad



Precisión: de los elementos seleccionados (aquellos predichos positivos), ¿qué porcentaje son relevantes (positivos reales)?

Exhaustividad: de los elementos relevantes (positivos reales), ¿qué porcentaje fueron seleccionados (predichos positivos)?

$$\text{Precision} = \frac{tp}{tp + fp} \quad 18 / (18 + 4) = 0.82$$

$$\text{Recall} = \frac{tp}{tp + fn} \quad 18 / (18 + 2) = 0.9$$

Atención: revisar la definición del orden de las filas y las columnas (o clase predecida y observada)

Clasificación Binaria • F-score

Dado un problema binario, proponer un test 100% exhaustivo y otro 100% preciso. ¿Son útiles?

Clasificación Binaria • F-score

Dado un problema binario, proponer un test 100% exhaustivo y otro 100% preciso. ¿Son útiles?

Problema: ninguna métrica por sí sola alcanza para evaluar el desempeño del test, ya que precisión y exhaustividad compiten entre sí.

Objetivo: encontrar un compromiso entre ambas métricas.

$$\text{F-SCORE} \longrightarrow F = 2 \times \frac{\text{precisión} \times \text{exhaustividad}}{\text{precisión} + \text{exhaustividad}}$$

Clasificación Multiclase

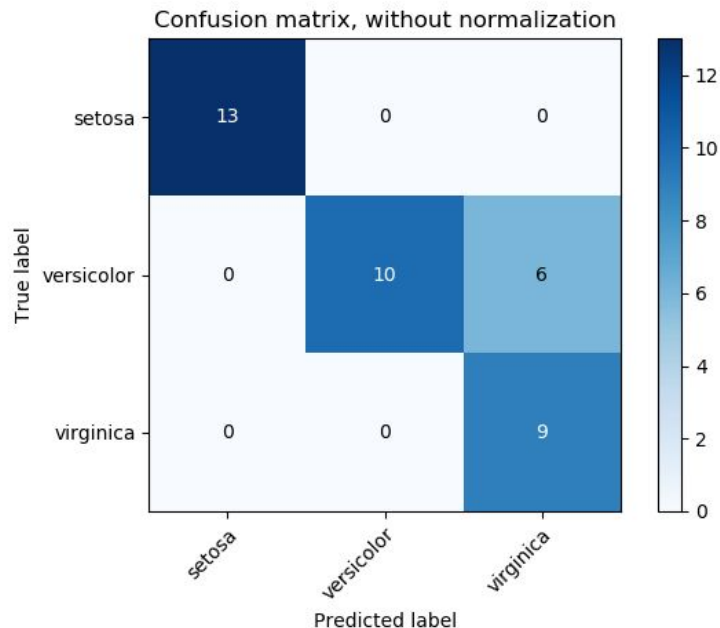
¿Cómo se generalizan los conceptos?

Verdaderos Positivos, Verdaderos Negativos, Falsos Positivos y Negativos: tengo que elegir una clase de interés.

Precisión y Exhaustividad: ídem, por clase.

Exactitud: Sigue valiendo.

¡La matriz de confusión sigue teniendo toda la información relevante!



Resumen

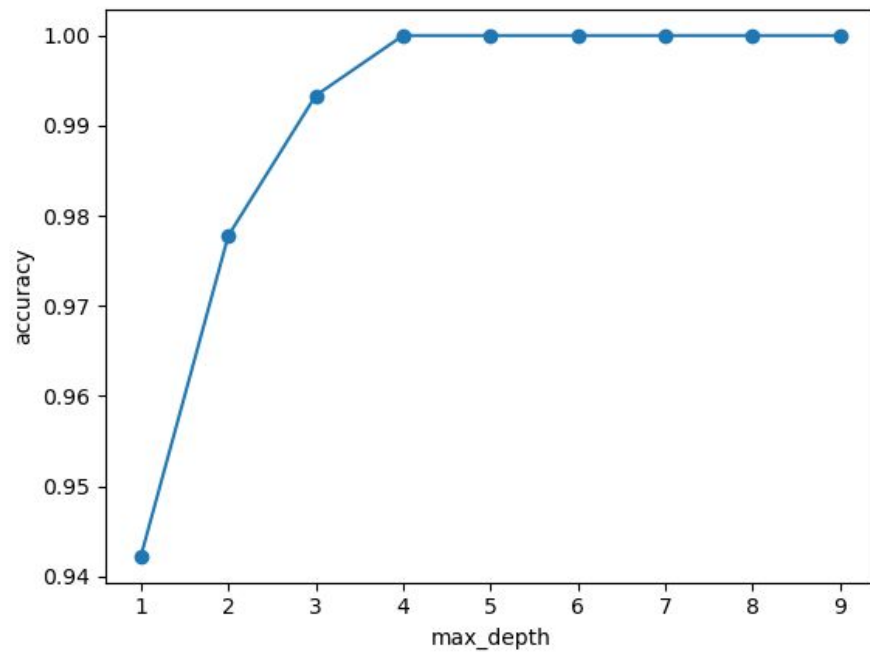
Wikipedia - Confusion matrix

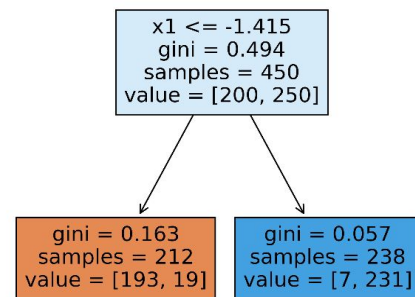
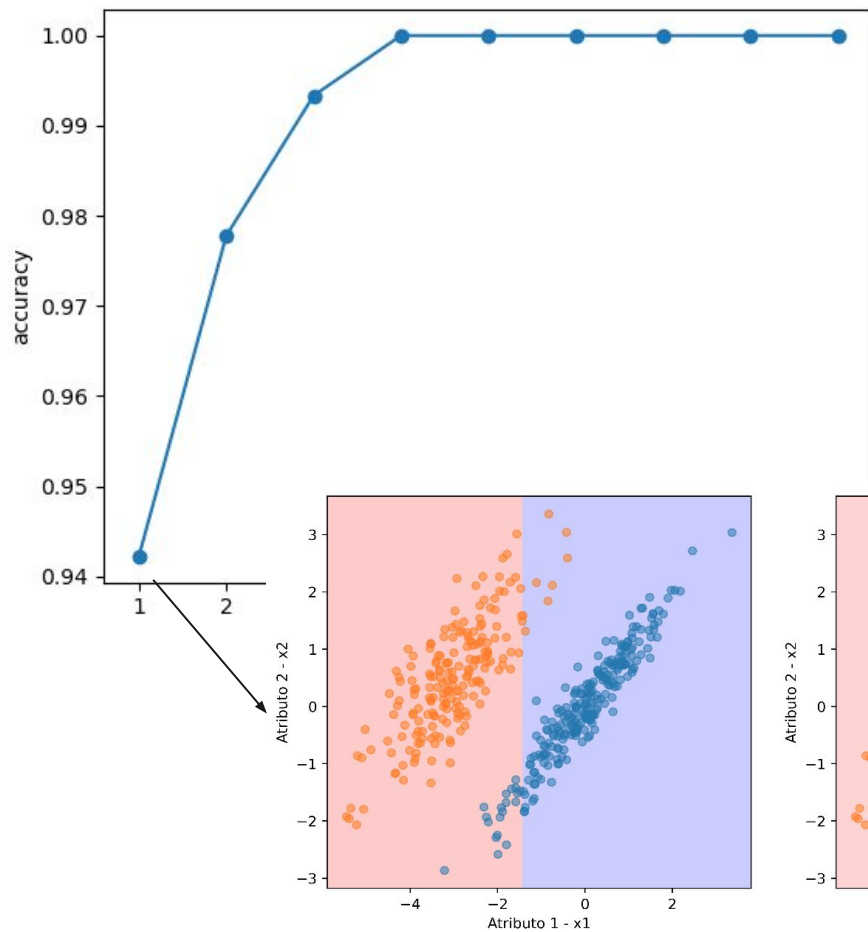
Sources: [12][13][14][15][16][17][18][19] view · talk · edit

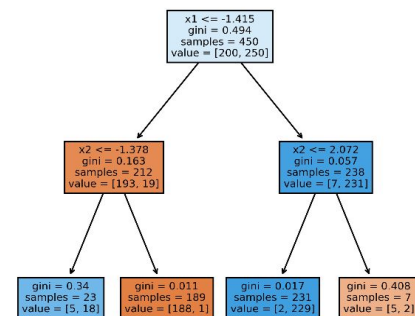
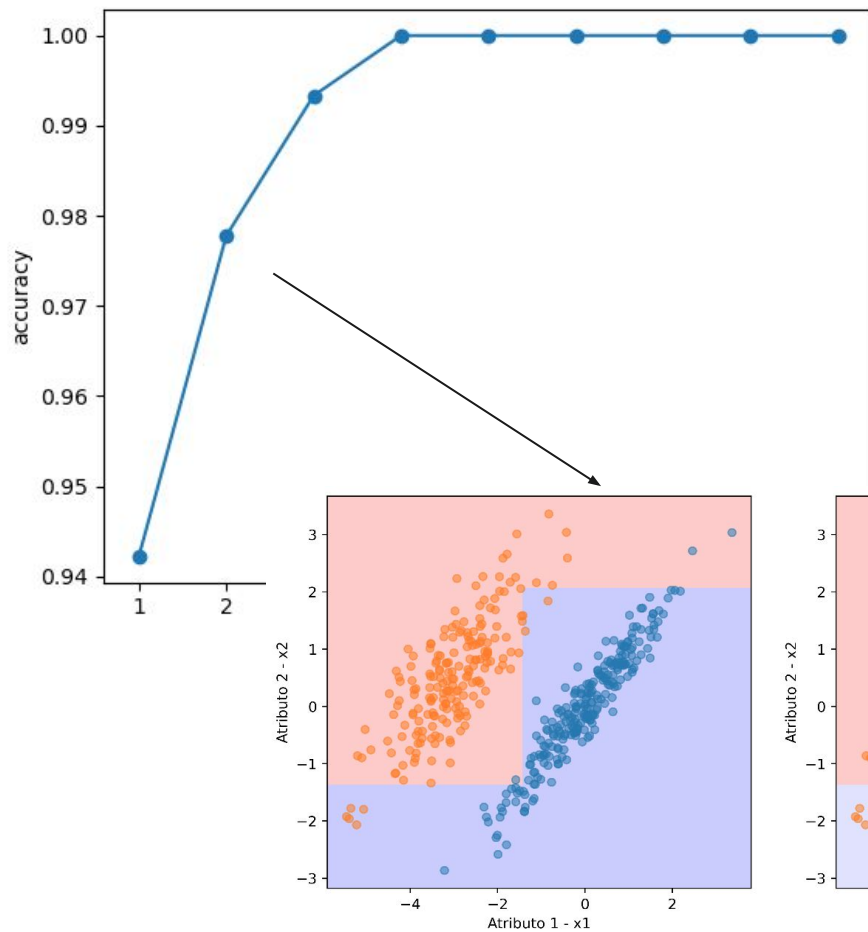
Predicted condition			Sources: [12][13][14][15][16][17][18][19] view · talk · edit		
	Predicted positive	Predicted negative	Informedness, bookmaker informedness (BM) = TPR + TNR − 1	Prevalence threshold (PT) $= \frac{\sqrt{\text{TPR} \times \text{FPR}} - \text{FPR}}{\text{TPR} - \text{FPR}}$	
Actual condition	Positive (P) [a]	False negative (FN), miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{\text{TP}}{\text{P}} = 1 - \text{FNR}$	False negative rate (FNR), miss rate type II error [c] $= \frac{\text{FN}}{\text{P}} = 1 - \text{TPR}$	
	Negative (N)[d]	True negative (TN), correct rejection[e]	False positive rate (FPR), probability of false alarm, fall-out type I error [f] $= \frac{\text{FP}}{\text{N}} = 1 - \text{TNR}$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{\text{TN}}{\text{N}} = 1 - \text{FPR}$	
	Prevalence $= \frac{\text{P}}{\text{P} + \text{N}}$	Positive predictive value (PPV), precision $= \frac{\text{TP}}{\text{TP} + \text{FP}} = 1 - \text{FDR}$	False omission rate (FOR) $= \frac{\text{FN}}{\text{TN} + \text{FN}} = 1 - \text{NPV}$	Positive likelihood ratio (LR+) $= \frac{\text{TPR}}{\text{FPR}}$	Negative likelihood ratio (LR-) $= \frac{\text{FNR}}{\text{TNR}}$
	Accuracy (ACC) $= \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$	False discovery rate (FDR) $= \frac{\text{FP}}{\text{TP} + \text{FP}} = 1 - \text{PPV}$	Negative predictive value (NPV) $= \frac{\text{TN}}{\text{TN} + \text{FN}} = 1 - \text{FOR}$	Markedness (MK), deltaP (Δp) = PPV + NPV − 1	Diagnostic odds ratio (DOR) $= \frac{\text{LR+}}{\text{LR-}}$
Balanced accuracy (BA) $= \frac{\text{TPR} + \text{TNR}}{2}$	F1 score $= \frac{2 \text{ PPV} \times \text{TPR}}{\text{PPV} + \text{TPR}}$ $= \frac{2 \text{ TP}}{2 \text{ TP} + \text{FP} + \text{FN}}$	Fowlkes– Mallows index (FM) $= \sqrt{\text{PPV} \times \text{TPR}}$	Matthews correlation coefficient (MCC) = $\sqrt{\text{TPR} \times \text{TNR} \times \text{PPV} \times \text{NPV}}$ · $\sqrt{\text{FNR} \times \text{FPR} \times \text{FOR} \times \text{FDR}}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{\text{TP}}{\text{TP} + \text{FN} + \text{FP}}$	

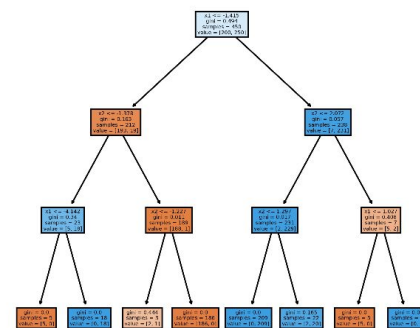
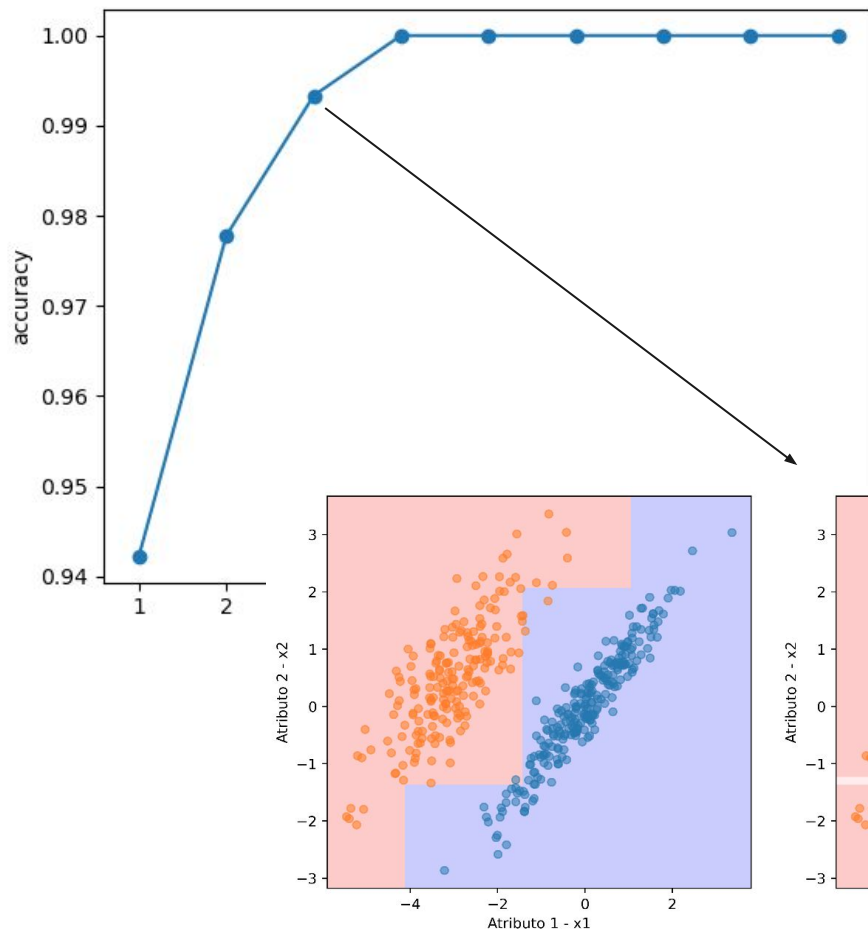
Evaluación de Modelos

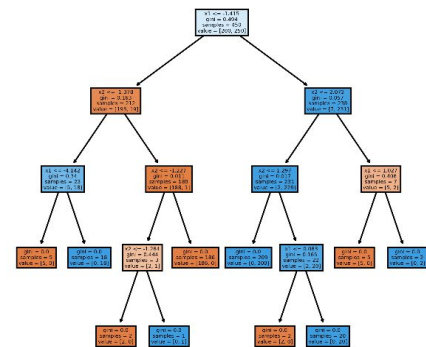
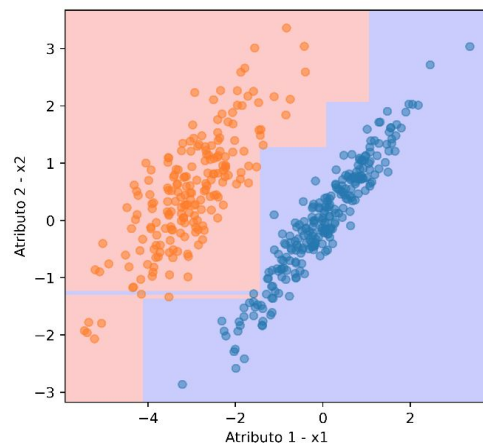
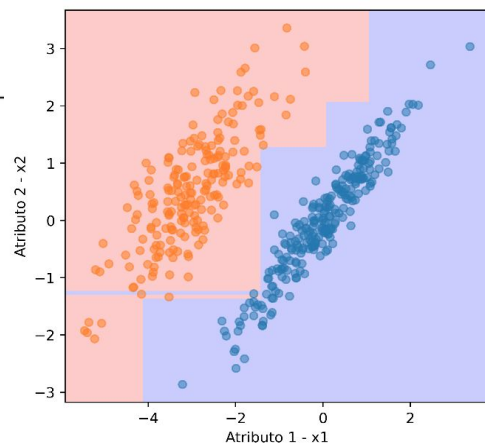
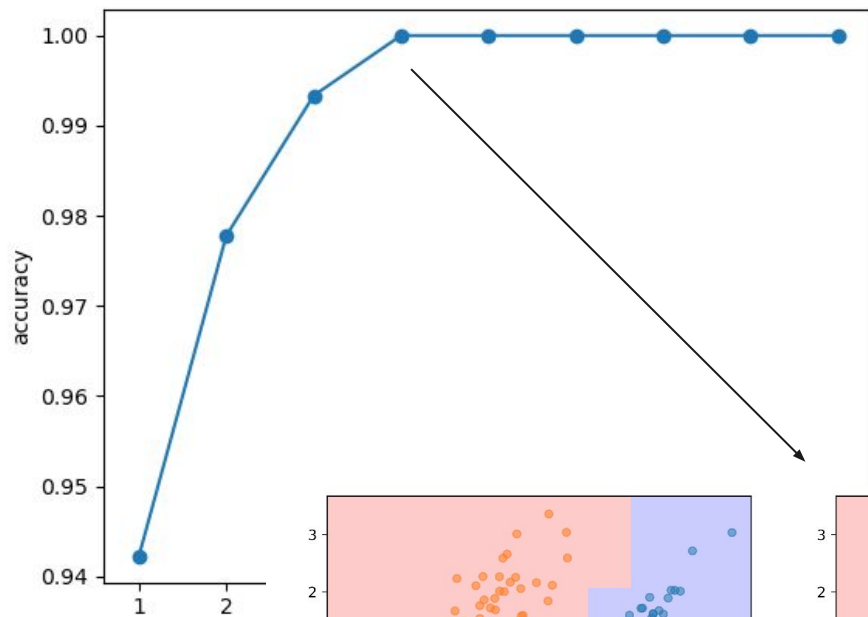
Repaso

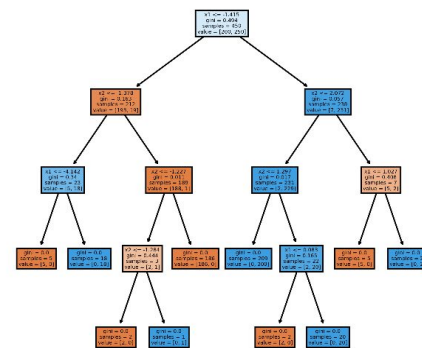
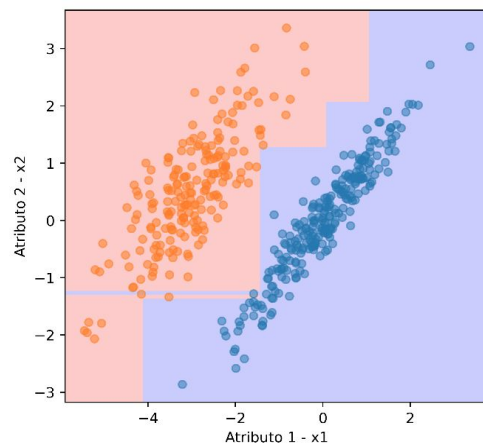
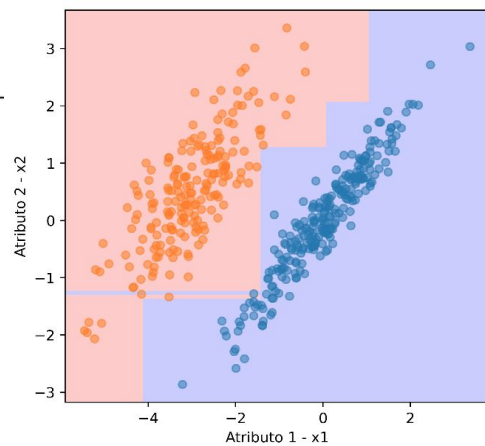
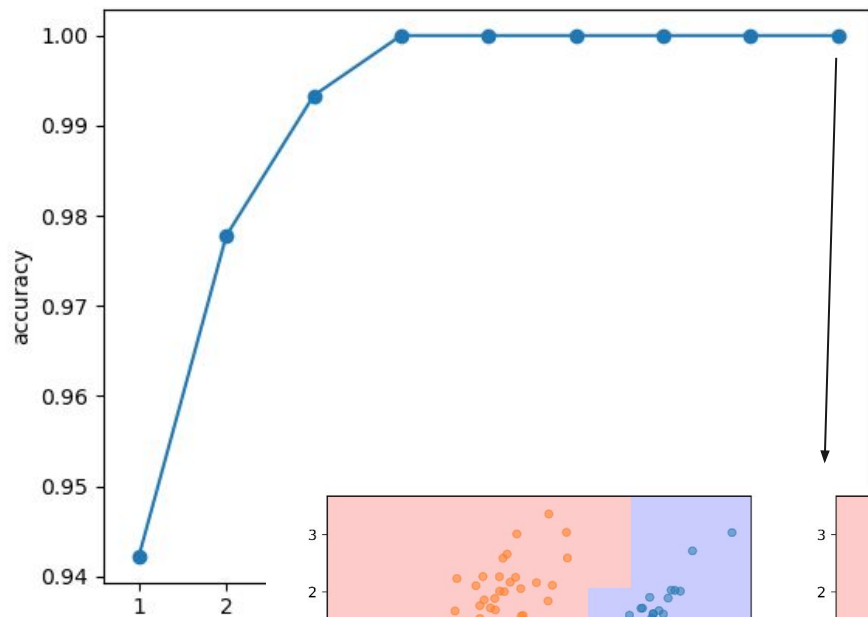




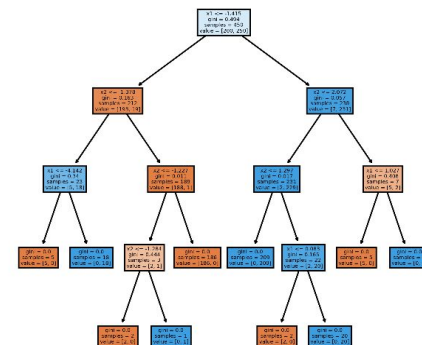
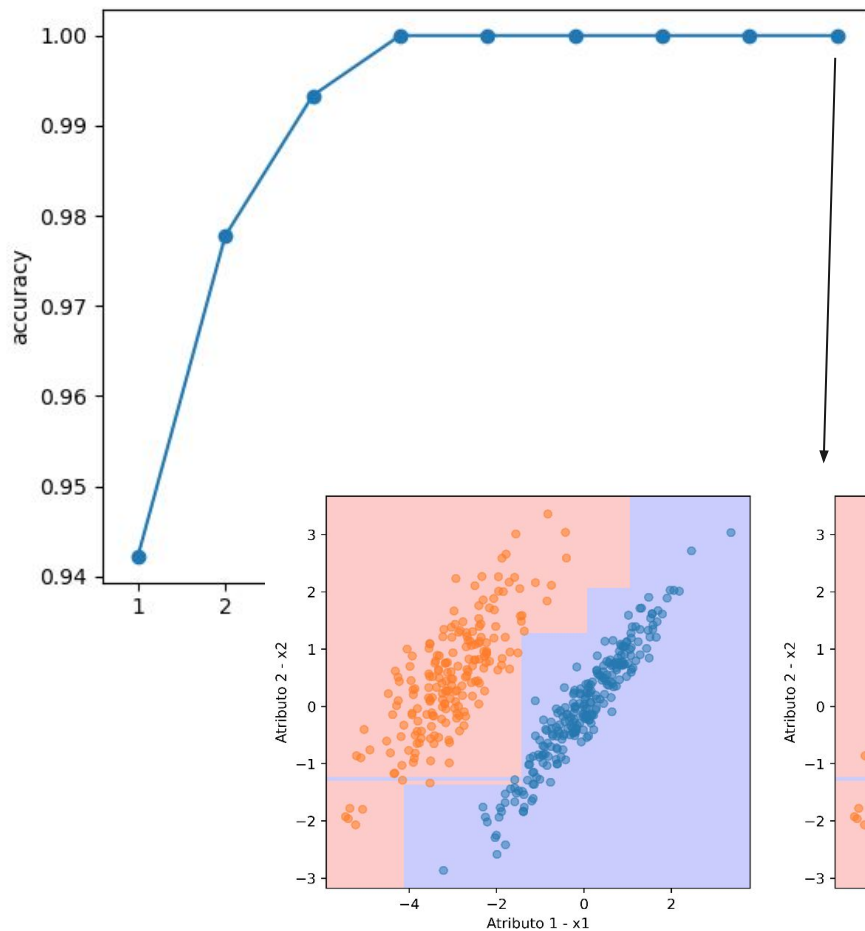




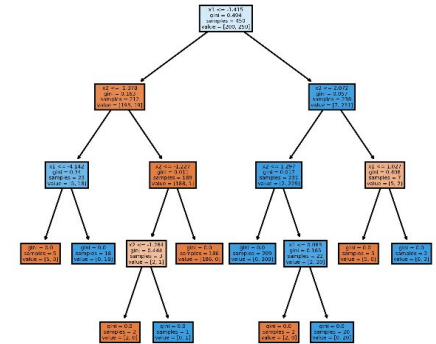
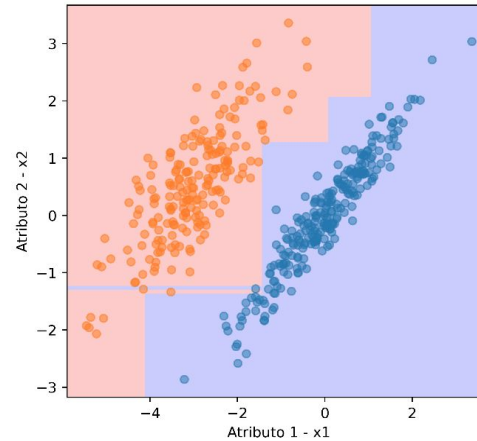
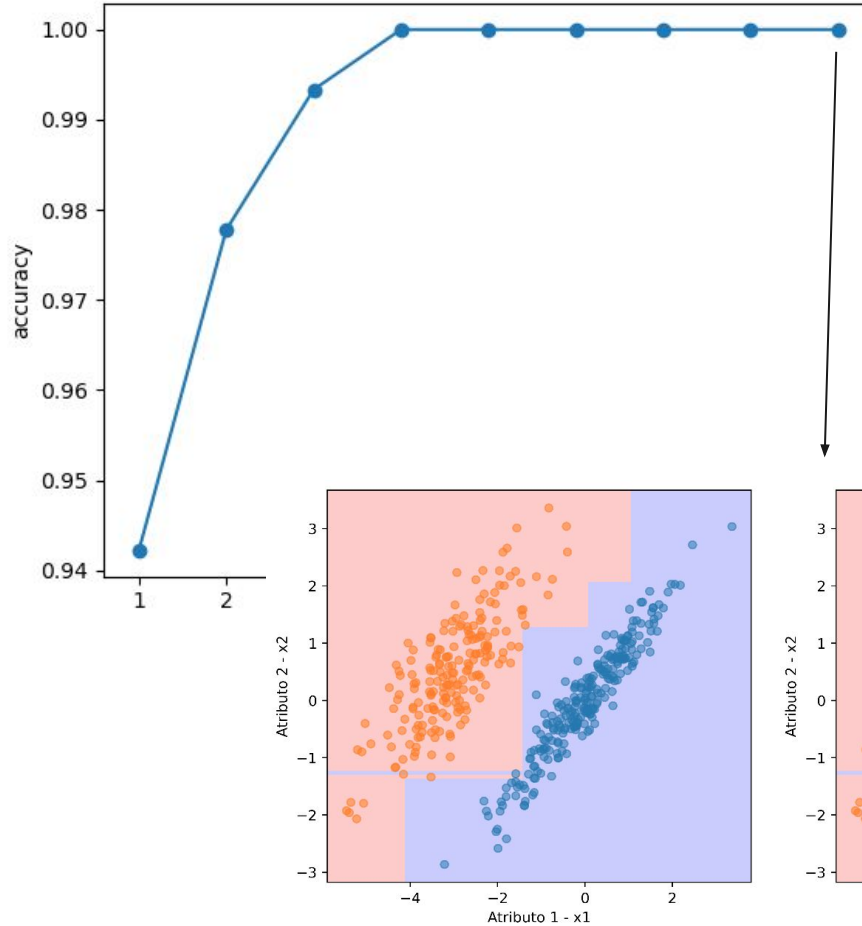




Es sospechoso que, con la suficiente profundidad, podemos obtener un 100% de exactitud.



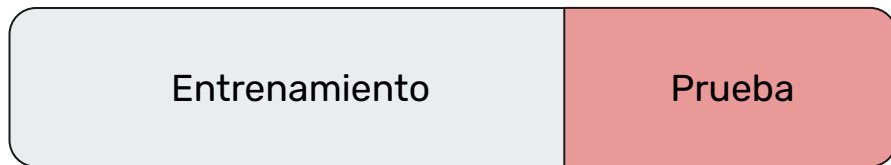
**Es mala idea medir el
desempeño de un modelo
sobre los mismos datos que
fue entrenado.**



Primera herramienta: train-test split

Puede ocurrir que el modelo se aprenda “de memoria” los datos con los que fue entrenado, por lo que **es importante evaluarlo con datos que sean independientes de los usados durante el entrenamiento.**

¿Solución? Train-test split.



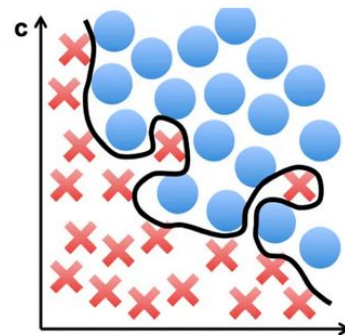
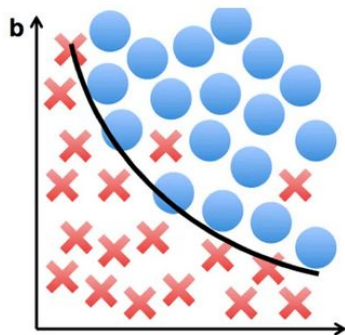
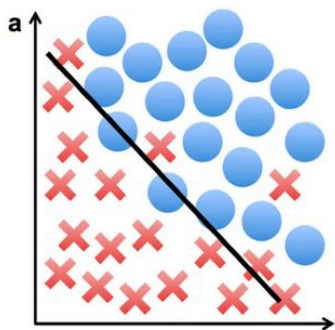
Entonces, antes de entrenar - o incluso aún más temprano en el flujo de trabajo - separamos los datos en dos conjuntos. Una parte la usaremos para entrenar el modelo, mientras que la otra para evaluarlo.

En general, los datos se separan al azar para evitar cualquier orden o estructura subyacente en los datos¹.

¹ En muchas situaciones, esto no es válido. Por ejemplo, cuando queremos entrenar un modelo que haga predicciones a futuro, o cuando trabajamos con varios pacientes y hacemos un seguimiento en el tiempo.

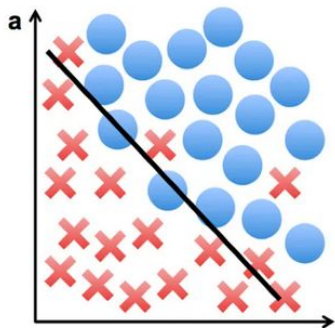
Consideremos los siguientes escenarios

¿Cuál clasificador les parece mejor?

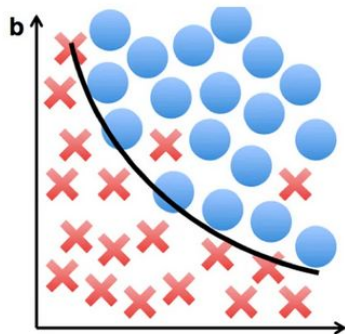


Consideremos los siguientes escenarios

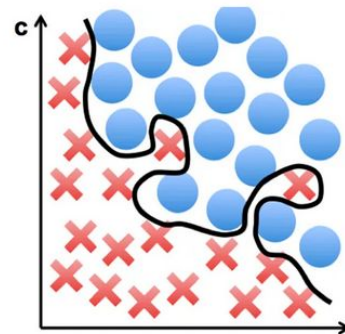
¿Cuál clasificador les parece mejor?



El **clasificador a** es muy simple y no reproduce correctamente la frontera entre las clases. Llamaremos **subajuste (underfitting)** a esta situación.

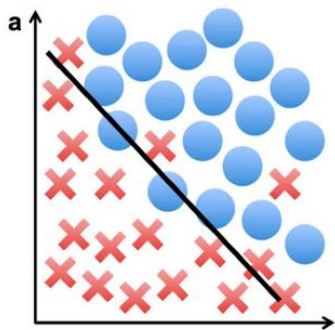


El **clasificador b** tiene la complejidad suficiente para encontrar una frontera que parece apropiada para estos datos.



El **clasificador c** parece muy flexible y se adaptó demasiado a los datos con los que fue entrenado. Llamaremos **sobreajuste (overfitting)** a esta situación.

Diagnóstico con train-test split

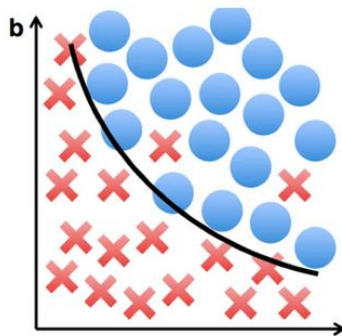


subajuste (underfitting)

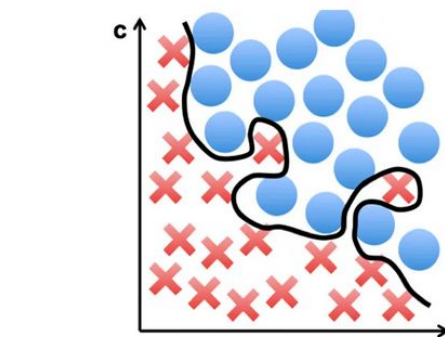
Desempeño en entrenamiento

$\approx$

Desempeño en evaluación



Ajuste correcto



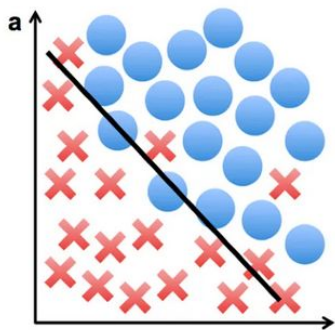
sobreajuste (overfitting)

Desempeño en entrenamiento

$\gg$

Desempeño en evaluación

Diagnóstico con train-test split

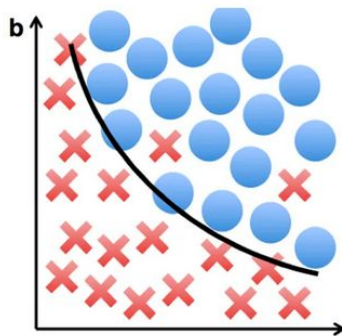


subajuste (underfitting)

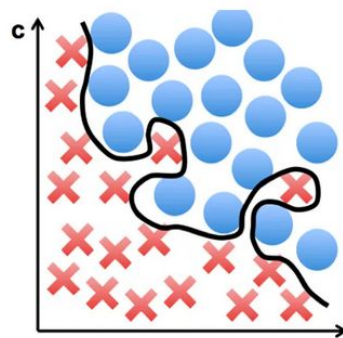
Desempeño en entrenamiento

$\approx$

Desempeño en evaluación



Ajuste correcto



sobreajuste (overfitting)

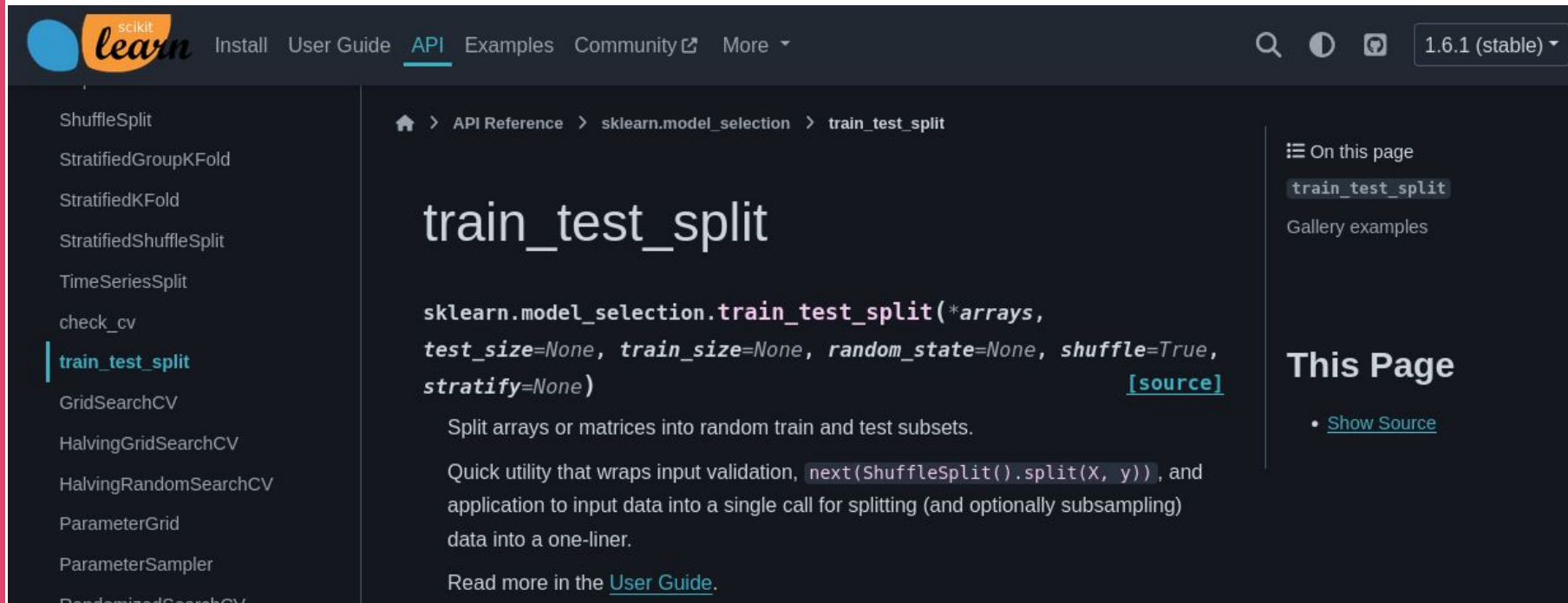
Desempeño en entrenamiento

$\gg$

Desempeño en evaluación

- Subajuste suele ser más difícil de diagnosticar, pero también menos grave (existe un modelo que podría andar mejor, pero no significa que el que tengamos ande mal).
- El gran enemigo suele ser el sobreajuste.

Código



The screenshot shows the scikit-learn website's API documentation for the `train_test_split` function. The top navigation bar includes links for 'Install', 'User Guide', 'API' (which is highlighted), 'Examples', 'Community', and 'More'. The version '1.6.1 (stable)' is displayed in the top right. On the left sidebar, a list of modules is shown, with `train_test_split` selected and highlighted in blue. The main content area features the breadcrumb path 'API Reference > sklearn.model_selection > train_test_split', followed by the function name 'train_test_split' in a large font. Below this, the function signature is provided: `sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)`, with a '[source]' link to the right. A descriptive paragraph states: 'Split arrays or matrices into random train and test subsets. Quick utility that wraps input validation, `next(ShuffleSplit().split(X, y))`, and application to input data into a single call for splitting (and optionally subsampling) data into a one-liner. Read more in the [User Guide](#).' On the right side, there is a section 'On this page' containing a link to 'train_test_split' and 'Gallery examples', and a 'This Page' section with a 'Show Source' link.

scikit-learn

Install User Guide **API** Examples Community More

1.6.1 (stable)

ShuffleSplit
StratifiedGroupKFold
StratifiedKFold
StratifiedShuffleSplit
TimeSeriesSplit
check_cv
train_test_split
GridSearchCV
HalvingGridSearchCV
HalvingRandomSearchCV
ParameterGrid
ParameterSampler
RandomizedSearchCV

API Reference > sklearn.model_selection > train_test_split

train_test_split

```
sklearn.model_selection.train_test_split(*arrays,  
test_size=None, train_size=None, random_state=None, shuffle=True,  
stratify=None)
```

[\[source\]](#)

Split arrays or matrices into random train and test subsets.

Quick utility that wraps input validation, `next(ShuffleSplit().split(X, y))`, and application to input data into a single call for splitting (and optionally subsampling) data into a one-liner.

Read more in the [User Guide](#).

On this page

train_test_split

Gallery examples

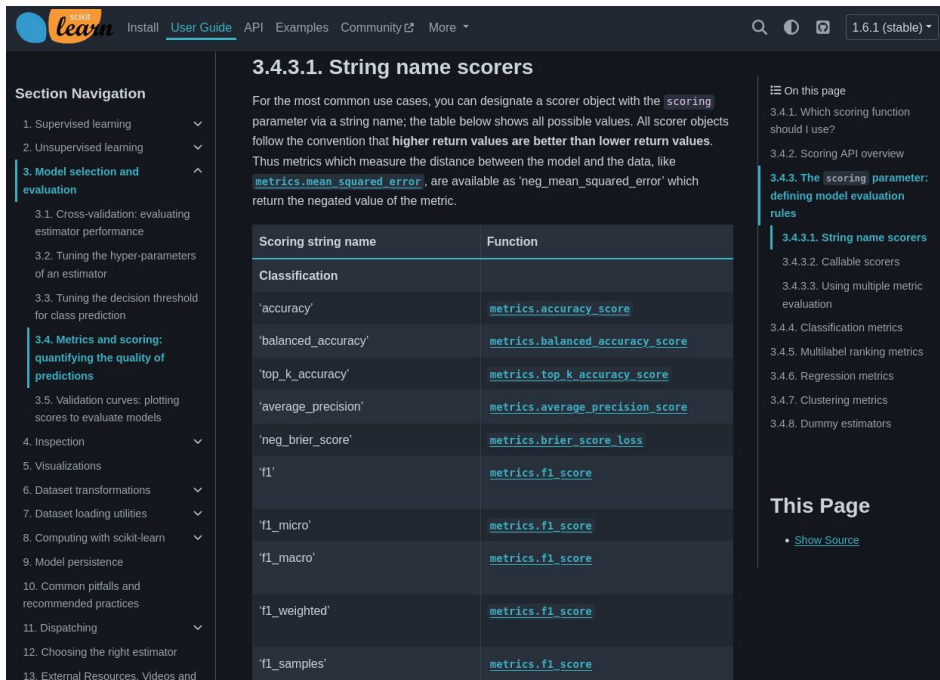
This Page

- [Show Source](#)



[train_test_split](#)

Código



The screenshot shows the scikit-learn User Guide page for 'String name scorers'. The page is dark-themed and includes a sidebar with 'Section Navigation' and a main content area with a table of scorers.

Section Navigation

- 1. Supervised learning
- 2. Unsupervised learning
- 3. Model selection and evaluation
 - 3.1. Cross-validation: evaluating estimator performance
 - 3.2. Tuning the hyper-parameters of an estimator
 - 3.3. Tuning the decision threshold for class prediction
 - 3.4. Metrics and scoring: quantifying the quality of predictions**
 - 3.5. Validation curves: plotting scores to evaluate models
- 4. Inspection
- 5. Visualizations
- 6. Dataset transformations
- 7. Dataset loading utilities
- 8. Computing with scikit-learn
- 9. Model persistence
- 10. Common pitfalls and recommended practices
- 11. Dispatching
- 12. Choosing the right estimator
- 13. External Resources, Videos and

3.4.3.1. String name scorers

For the most common use cases, you can designate a scorer object with the `scoring` parameter via a string name; the table below shows all possible values. All scorer objects follow the convention that **higher return values are better than lower return values**. Thus metrics which measure the distance between the model and the data, like `metrics.mean_squared_error`, are available as 'neg_mean_squared_error' which return the negated value of the metric.

Scoring string name	Function
Classification	
'accuracy'	metrics.accuracy_score
'balanced_accuracy'	metrics.balanced_accuracy_score
'top_k_accuracy'	metrics.top_k_accuracy_score
'average_precision'	metrics.average_precision_score
'neg_brier_score'	metrics.brier_score_loss
'f1'	metrics.f1_score
'f1_micro'	metrics.f1_score
'f1_macro'	metrics.f1_score
'f1_weighted'	metrics.f1_score
'f1_samples'	metrics.f1_score

On this page

- 3.4.1. Which scoring function should I use?
- 3.4.2. Scoring API overview
- 3.4.3. The `scoring` parameter: defining model evaluation rules**
 - 3.4.3.1. String name scorers**
 - 3.4.3.2. Callable scorers
 - 3.4.3.3. Using multiple metric evaluation
- 3.4.4. Classification metrics
- 3.4.5. Multilabel ranking metrics
- 3.4.6. Regression metrics
- 3.4.7. Clustering metrics
- 3.4.8. Dummy estimators

This Page

- [Show Source](#)



Metrics and scoring: quantifying the quality of predictions - [String name scorers](#)